23F-0577
BSCS-7D
Ass #2
DLP

Question 1

Task A

1. AlexNet. As written 11×11 convolutional at start which is used by AlexNet
2. VGG. As it uses multiple 3×3 filters to make it deeper and efficient.
3. VGG. as talking about increasing feature map to reduce spatial Resolution but it is also used in ResNet.
4. ResNet. ResNet allows shortcut connection. which make information and gradient to flow directly b/w layers.
5. Inception. The inception module processes same input through different branches such as 1×1, 3×3, 5×5
6. Same Inception. Global average pooling converts each feature map into single value it reduces need of large fully connected layers

3. Main Problem motivating each Innovation.

1- AlexNet

Other methods struggled with large scale image classification and automatic feature learning.

Introduce a successful deep CNN using ReLU, dropout and large convolutional filter.

2- VGG

Large convolutional filters required more parameters and computation.

Use repeated 3×3 filters to build deeper network with smaller filters

3- ResNet.

Increasing network depth causes losing gradient makes a deep network difficult to train

Introduce shortcut connection to improve information and gradient flow

4- InceptionNet

A single filter size cannot efficiently capture features at different scales. while increasing network increases computation.

Introduce parallel branches with different filter size.

CamScanner

5- EfficientNet:

Increasing only depth width or image resolution does not provide efficient balance b/w cost and accuracy.

Introduce compound scaling which scales depth width and input resolution together in balance way.

C- Compare AlexNet and Inception

Aspect	AlexNet	Inception
Network depth	Relatively shallow	Deeper
Filter design	use large filter usually 11×11 at beginning	Uses multiple filter size like 1×1, 3×3 and 5×5
Feature extraction	mainly sequential	Parallel
Information flow	Layer by Layer	multiple branches process same input
compuational Efficiency	Large filters and fully connected layers requires significant computation	use 1×1 convolution to reduce computation and parameters
Main Innovation	Power of deep CNN for large scale image classification	Inception module with parallel branches

D- Plain Network VS Network with shortcut.

In plan network input must pass through every convolutional layer. As network become deeper the gradient can become very small during backpropagation. This makes earlier layer difficult to train and can cause vanishing gradient.

In shortcut network some layers are connected to da later layer through shortcut connection. These connection provide direct path for information during forward/backward propagation.

Therefore shortcut network is easier to train and achieve better performance. Even though it has same convolutional layer.

E. 7×7 VS 3×3 Convolutional

We can use multiple 3×3 features instead of 7×7 Because.

3×3 filters required fewer parameters than a 7×7

multiple 3×3 layers allow the network to learn more complex features.

Each layer can apply an activation function, increases networks ability to learn complex pattern.

why does not Inception use only 3×3

Because different features are used by different filters.

Inception processes same input through different branches.

Input 1×1

Input 3×3

Input 5×5

Input pooling

Then their outputs are combined.

F- Chronological Order AlexNet and Inception,

1- AlexNet 2012

It introduce high performance on large scale image classification.

It introduce

① Relu activation

② GPU based training

③ Dropout

④ convolutional layer

⑤ Pooling.

Problem Addressed.

How efficiently learn image by large scale image classification,

2- Inception 2014

After CNNs become deeper and more powerful, increasing size of network also increased.

Inception uses multiple branches with different operation and combination

It also used 1×1 convolutions to reduce number of channels.

main problem addressed

How can a CNN become more powerful and deeper while keeping computation and number of parameters manageable.

CamScanner

Question 2 Alex Net Architecture

A) Trainable parameters of convolutional layer.

$C_{out} * (C_{in} * k * k) + C_b$

C_b for biases.

Conv1

$C_{in} = 3$

$C_{out} = 96$

kernel $= 11 * 11$

Parameters $= 96(3 * 11 * 11) + 96$

$= 34848 + 96$

$= 34944$

Conv 2

$C_{in} = 96$

$C_{out} = 256$

kernel $= 5 * 5$

Parameter $= 256(96 * 5 * 5) + 256$

$= 614,400 + 256$

$= 614,656$

Conv 3

$C_{in} = 256$

$C_{out} = 384$

kernel $3 * 3$

$= 384(256 * 3 * 3) + 384$

$= 884,736 + 384$

$= 885120$

Conv4

C_{in} 384

$C_{out} = 384$

kernel $3 * 3$

$= 384(384 * 3 * 3) + 384$

$= 1327104 + 384$

$= 1327488$

Conv5

$C_{in} = 384$

$C_{out} = 256$

kernel $= 3 * 3$

$= 256(384 * 3 * 3) + 256$

$= 884736 + 256$

$= 884992$

5- FC1 - 120 neurons

Before FC1 we flatten

$5 \times 5 \times 16 = 400$

So FC1 has

400 input

120 output

parameter = $(400 \times 120) + 120$

$= 48000 + 120$

$= 48120$

Total parameter = $48120 + 2572$

$= 50692$

6- FC2 - 84 neurons

Input 120

output 84

Parameters = $(120 + 84) + 84$

$= 10080 + 84$

$= 10164$

Total parameters = $50692 + 10164$

$= 60856$

7- FC3 10 neuron + Softmax.

Input 84

output 10

parameter = $(84 + 10) + 10$

$= 840 + 10$

$= 850$

Softmax just converts 10 output neurons to probability so no trainable parameters

Softmax parameters = 0

Total parameter = $60856 + 850$

$= 61706$

Total = 61706

Question #3

1- Conv 1

Input = 32×32×1
Kernel = 5×5
Number of filters = 6

So

parameters = (5×5×1×6) + 6
= 150 + 6
= 156

output 28×28×6

2. Average pooling 1

filters = 2×2
Stride = 2

pooling has no weights and biases so no parameters.

output 14×16×6

156+0

Total parameters = 156

3- Conv 2

Input 14×14×6
Kernel = 5×5
Input channels = 6
filters = 16
= (5×5×6×16) + 16
= 2400 + 16
= 2416

output 10×10×16

Total parameter = 156 + 2416 = 2572

4. Average pooling 2

Filter 2×2
Stride 2
pooling has no weights and biases so no parameters
output 5×5×16

Total = 2572+0 = 2572

B- Output Size After Convolution and Pooling.

Pool 1: 3×3 stride 2
Pool 2: 3×3 stride 2
Pool 5: 3×3 stride 2

$$\text{Output} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Input

227×227×3

Conv 1

kernel = 11
Stride 4
padding 0
pooling 3
96 filters

~~227 = 11+2(0)~~

$$\text{output} = \left\lfloor \frac{227 - 11 + 2(0)}{4} \right\rfloor + 1$$

= 55×55×96

Pool 1

$$\text{output} = \left\lfloor \frac{55 - 3 + 2(0)}{2} \right\rfloor + 1$$

= 27×27×96

Conv 2

256 filters
k = 5×5
padding 2

$$\text{output} = \left\lfloor \frac{27 - 5 + 2(2)}{1} \right\rfloor + 1$$

= 24+1 = 27

= 27×27×256

Pool 2

$$\text{output} = \left\lfloor \frac{27 - 3 + 2(0)}{2} \right\rfloor + 1$$

= 13×13×256

Conv 3

$$\left\lfloor \frac{13 - 3 + 2(1)}{1} \right\rfloor + 1$$

384 filters
k = 3×3
padding 1

= 13×13×384

Conv 4

k 3×3 384 filters
padding 1

$$\text{output} = \left\lfloor \frac{13 - 3 + 2(1)}{1} \right\rfloor + 1$$

= 13×13×384

Conv 5

$$\text{output} = \left\lfloor \frac{13 - 3 + 2(1)}{1} \right\rfloor + 1$$

= 12+1 = 13

= 13×13×256

k 3×3
filter 256
pad 1

Pool 5

$$\text{output} = \left\lfloor \frac{13 - 3 + 2(0)}{2} \right\rfloor + 1$$

= 6×6×256

C) Fully Connected parameters

FC1

Input = 9216 Biases = 4096

Output = 4096

parameters = (9216 * 4096) + 4096

= 37752832

FC2

Input = 4096 Biases = 4096

Output = 4096

(4096 * 4096) + 4096

= 16781312

FC3

Input 4096 Biases = 1000

output 1000

parameters = (4096 * 1000) + 1000

= 4097000

D. Total Trainable parameters.

convolutional total.

= 34944 + 614656 + 1327488 + 884992 + 885120

= 3947200

Fully Connected Total

= 37752832 + 16781312 + 4097000

= 58631144

Grand Total.

= 3947200 + 58631144

= 62378344